

Error Handling

Link: <https://docs.mulesoft.com/mule-runtime/latest/mule-error-concept>

Link: <https://mulesy.com/error-handling-in-mule-4/>

- If there is no error handler defined, a **Mule default error handler** is used
 - Implicitly and globally handles all messaging errors thrown in Mule applications
 - Stops execution of the flow and logs **information** about the error
 - Cannot be configured
- When an error is thrown, an **error object** is created
- Two of its properties include
 - **error.description** – a string
 - **error.errorType** – an object
- Error types are identified by a namespace and an identifier
 - HTTP:UNAUTHORIZED, HTTP:CONNECTIVITY, VALIDATION:INVALID_BOOLEAN

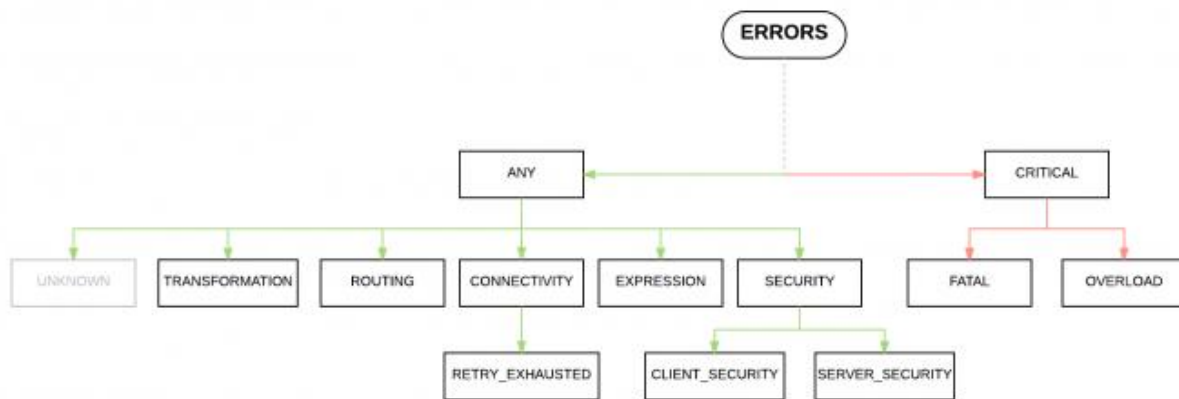
Example for error.description:

>>> Invalid response (string)

Example for error.errorType:

```
{
  "parentErrorType": {
    "parentErrorType": {
      "parentErrorType": {
        "parentErrorType": null,
        "namespace": "MULE",
        "identifier": "ANY"
      },
      "namespace": "MULE",
      "identifier": "VALIDATION"
    },
    "namespace": "VALIDATION",
    "identifier": "VALIDATION"
  },
  "namespace": "VALIDATION",
  "identifier": "INVALID_BOOLEAN"
}
```

Error type:



>>> Any errors can be handled, but critical errors cannot be handled.

Error handling scopes:

- **On Error Propagate**
 - All processors in the error handling scope are executed
 - At the end of the scope
 - The rest of the flow that threw the error is not executed
 - *The error is rethrown up to the next level and handled there*
 - An HTTP Listener returns an **error** response
- **On Error Continue**
 - All processors in the error handling scope are executed
 - At the end of the scope
 - The rest of the flow that threw the error is not executed
 - *The event is passed up to the next level as if the flow execution had completed successfully*
 - An HTTP Listener returns a **successful** response